

Beautiful Soup

Web scraping is a useful technique for extracting information from the internet, especially when the amount of data is huge. If you're scraping websites using Python language, you should learn how to use BeautifulSoup. It is one of the most popular libraries for web scraping in Python.

Steps involved in web scraping

- **Send an HTTP Request:** Use the requests library to send a request to the webpage URL and get the HTML content in response.
- **Parse the HTML Content:** Use a parser like html.parser or html5lib to convert the raw HTML into a structured format (parse tree).
- **Extract Data:** Use BeautifulSoup to navigate the parse tree and extract the required data using tags, classes, or IDs.

It transforms complex HTML/XML documents into a Python object tree and provides simple methods to help you navigate, search, and modify the tree:

- Navigate - Down (.head, .title, .body, etc.), up (.parent, .parents), sideways (.next_sibling, .previous_sibling, .etc.), back and forth (.next_element, .previous_element, etc.)
- Search - find_all(), find(), find_next(), etc.
- Modify - append(), extend(), insert(), clear(), etc.

This imports the necessary libraries: BeautifulSoup from the bs4 package, which is used for parsing HTML and XML documents, and requests, which is used for making HTTP requests to fetch web page content. Defining the URL.

```
In [2]: from bs4 import BeautifulSoup
import requests
url="https://pandas.pydata.org/"
```

This line uses the requests.get() function to send an HTTP GET request to the specified url. The response from the server is stored in the req variable. This req object now contains information about the response, including the status code, headers, and the raw content of the web page.

The first step in web scraping is to send an HTTP request to the target webpage and fetch its raw HTML content. This is done using requests library.

```
In [9]: req = requests.get(url)
        print(req.text)
```

```
        role="button"
        aria-haspopup="true"
        aria-expanded="false">About us</a>
<div class="dropdown-menu" aria-labelledby="about-us">

    <a class="dropdown-item" href="about/">About pandas</a>

    <a class="dropdown-item" href="about/roadmap.html">Project roadmap</a>

    <a class="dropdown-item" href="about/governance.html">Governance</a>

    <a class="dropdown-item" href="about/team.html">Team</a>

    <a class="dropdown-item" href="about/sponsors.html">Sponsors</a>

    <a class="dropdown-item" href="about/citing.html">Citing and logo</a>

</div>
</li>
```

To actually parse and extract data from the HTML content, you would typically proceed by creating a BeautifulSoup object:

```
In [4]: soup = BeautifulSoup(req.text, "html.parser")
        print(soup.title)

<title>pandas - Python Data Analysis Library</title>
```

```
In [4]: for link in soup.find_all('a'):
        print(link.get('href'))
```

```
.  
#  
about/  
about/roadmap.html  
about/governance.html  
about/team.html  
about/sponsors.html  
about/citing.html  
getting_started.html  
docs/  
#  
community/blog/  
https://stackoverflow.com/questions/tagged/pandas (https://stackoverflow.com/questions/tagged/pandas)  
community/coc.html  
community/ecosystem.html  
community/benchmarks.html  
contribute.html  
https://www.python.org (https://www.python.org)  
getting_started.html  
getting_started.html  
docs/getting_started/index.html  
docs/user_guide/index.html  
docs/reference/index.html  
docs/development/index.html  
docs/whatsnew/index.html  
about/index.html  
https://stackoverflow.com/questions/tagged/pandas (https://stackoverflow.com/questions/tagged/pandas)  
community/ecosystem.html  
https://numfocus.org/ (https://numfocus.org/)  
https://www.twosigma.com/ (https://www.twosigma.com/)  
https://voltrondata.com/ (https://voltrondata.com/)  
https://www.coiled.io (https://www.coiled.io)  
https://quansight.com/ (https://quansight.com/)  
https://www.nvidia.com (https://www.nvidia.com)  
https://www.intel.com/ (https://www.intel.com/)  
https://tidelift.com (https://tidelift.com)  
https://chanzuckerberg.com/ (https://chanzuckerberg.com/)  
https://www.bodo.ai/ (https://www.bodo.ai/)  
about/sponsors.html  
docs/whatsnew/v2.2.2.html  
docs/  
https://github.com/pandas-dev/pandas/releases/download/v2.2.2/pandas-2.2.2.tar.gz (https://github.com/pandas-dev/pandas/releases/download/v2.2.2/pandas-2.2.2.tar.gz)  
https://t.me/s/pandas\_dev (https://t.me/s/pandas\_dev)  
https://fosstodon.org/@pandas\_dev (https://fosstodon.org/@pandas\_dev)  
https://twitter.com/pandas\_dev/ (https://twitter.com/pandas\_dev/)  
https://amzn.to/3DyLaJc (https://amzn.to/3DyLaJc)  
https://store.metasnake.com/effective-pandas-book/lhte7 (https://store.metasnake.com/effective-pandas-book/lhte7)  
https://pandas.pydata.org/pandas-docs/stable/whatsnew/v2.1.4.html (https://pandas.pydata.org/pandas-docs/stable/whatsnew/v2.1.4.html)  
https://pandas.pydata.org/pandas-docs/version/2.1.4/ (https://pandas.pydata.org/pandas-docs/version/2.1.4/)
```

<https://github.com/pandas-dev/pandas/releases/download/v2.1.4/pandas-2.1.4.tar.gz> (<https://github.com/pandas-dev/pandas/releases/download/v2.1.4/pandas-2.1.4.tar.gz>)
<https://pandas.pydata.org/pandas-docs/stable/whatsnew/v2.0.3.html> (<https://pandas.pydata.org/pandas-docs/stable/whatsnew/v2.0.3.html>)
<https://pandas.pydata.org/pandas-docs/version/2.0.3/> (<https://pandas.pydata.org/pandas-docs/version/2.0.3/>)
<https://github.com/pandas-dev/pandas/releases/download/v2.0.3/pandas-2.0.3.tar.gz> (<https://github.com/pandas-dev/pandas/releases/download/v2.0.3/pandas-2.0.3.tar.gz>)
<https://pandas.pydata.org/pandas-docs/stable/whatsnew/v1.5.3.html> (<https://pandas.pydata.org/pandas-docs/stable/whatsnew/v1.5.3.html>)
<https://pandas.pydata.org/pandas-docs/version/1.5.3/> (<https://pandas.pydata.org/pandas-docs/version/1.5.3/>)
<https://github.com/pandas-dev/pandas/releases/download/v1.5.3/pandas-1.5.3.tar.gz> (<https://github.com/pandas-dev/pandas/releases/download/v1.5.3/pandas-1.5.3.tar.gz>)
<https://pandas.pydata.org/pandas-docs/stable/whatsnew/v1.4.4.html> (<https://pandas.pydata.org/pandas-docs/stable/whatsnew/v1.4.4.html>)
<https://pandas.pydata.org/pandas-docs/version/1.4.4/> (<https://pandas.pydata.org/pandas-docs/version/1.4.4/>)
<https://github.com/pandas-dev/pandas/releases/download/v1.4.4/pandas-1.4.4.tar.gz> (<https://github.com/pandas-dev/pandas/releases/download/v1.4.4/pandas-1.4.4.tar.gz>)
#show-more-releases
<https://pandas.pydata.org/pandas-docs/stable/whatsnew/v1.3.5.html> (<https://pandas.pydata.org/pandas-docs/stable/whatsnew/v1.3.5.html>)
<https://pandas.pydata.org/pandas-docs/version/1.3.5/> (<https://pandas.pydata.org/pandas-docs/version/1.3.5/>)
<https://github.com/pandas-dev/pandas/releases/download/v1.3.5/pandas-1.3.5.tar.gz> (<https://github.com/pandas-dev/pandas/releases/download/v1.3.5/pandas-1.3.5.tar.gz>)
https://t.me/s/pandas_dev (https://t.me/s/pandas_dev)
https://fosstodon.org/@pandas_dev (https://fosstodon.org/@pandas_dev)
https://x.com/pandas_dev/ (https://x.com/pandas_dev/)
<https://github.com/pandas-dev/pandas/> (<https://github.com/pandas-dev/pandas/>)
<https://stackoverflow.com/questions/tagged/pandas> (<https://stackoverflow.com/questions/tagged/pandas>)
<https://numfocus.org> (<https://numfocus.org>)
<https://www.ovhcloud.com> (<https://www.ovhcloud.com>)

```
In [6]: html_doc = """
<html><head><title>The Dormouse's story</title></head>
<body>
<p class="title"><b>The Dormouse's story</b></p>
<p class="story">Once upon a time there were three little sisters; and their names
→were
<a href="http://example.com/elsie" class="sister" id="link1">Elsie</a>,
<a href="http://example.com/lacie" class="sister" id="link2">Lacie</a> and
<a href="http://example.com/tillie" class="sister" id="link3">Tillie</a>;
and they lived at the bottom of a well.</p>
<p class="story">...</p>
"""
```

This line creates a BeautifulSoup object.

html_doc: This variable is expected to contain the raw HTML content as a string. 'html.parser': This specifies the parser to be used. html.parser is a built-in Python HTML parser. Other parsers like lxml or html5lib can also be used if installed.

```
In [7]: from bs4 import BeautifulSoup
soup = BeautifulSoup(html_doc, 'html.parser')
```

The `prettify()` method in BeautifulSoup is used to format HTML or XML documents in a human-readable, indented structure. When called on a BeautifulSoup object (typically named `soup`), it returns a string representation of the parsed document with proper indentation and line breaks, making the structure of the HTML or XML content much clearer.

```
In [8]: print(soup.prettify())

<html>
<head>
  <title>
    The Dormouse's story
  </title>
</head>
<body>
  <p class="title">
    <b>
      The Dormouse's story
    </b>
  </p>
  <p class="story">
    Once upon a time there were three little sisters; and their names
    were
    <a class="sister" href="http://example.com/elsie" id="link1">
      Elsie
    </a>
    ,
    <a class="sister" href="http://example.com/lacie" id="link2">
      Lacie
    </a>
    and
    <a class="sister" href="http://example.com/tillie" id="link3">
      Tillie
    </a>
    ;
    and they lived at the bottom of a well.
  </p>
  <p class="story">
    ...
  </p>
</body>
</html>
```

If you want the tag, just say `soup.head`:

```
In [10]: soup.head
```

```
Out[10]: <head><title>The Dormouse's story</title></head>
```

```
In [11]: soup.title
```

```
Out[11]: <title>The Dormouse's story</title>
```

```
In [12]: soup.title.name
```

```
Out[12]: 'title'
```

```
In [13]: soup.title.string
```

```
Out[13]: "The Dormouse's story"
```

```
In [14]: soup.title.parent.name
```

```
Out[14]: 'head'
```

```
In [15]: soup.p
```

```
Out[15]: <p class="title"><b>The Dormouse's story</b></p>
```

```
In [17]: for link in soup.find_all('p'):  
         print(link.text)
```

```
The Dormouse's story  
Once upon a time there were three little sisters; and their names  
were  
Elsie,  
Lacie and  
Tillie;  
and they lived at the bottom of a well.  
...
```

In BeautifulSoup, `soup.p['class']` is used to access the value of the class attribute of the first tag found in the parsed HTML document represented by the soup object.

```
In [18]: soup.p['class']
```

```
Out[18]: ['title']
```

Using a tag name as an attribute will give you only the first tag by that name:

```
In [19]: soup.a
```

```
Out[19]: <a class="sister" href="http://example.com/tillie" id="link3">Tillie</a>
```

If you need to get all the tags, or anything more complicated than the first tag with a certain name, you'll need to

use one of the methods described in Searching the tree, such as `find_all()`:

```
In [25]: soup.find_all('a')
```

```
Out[25]: [<a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>,
<a class="sister" href="http://example.com/lacie" id="link2">Lacie</a>,
<a class="sister" href="http://example.com/tillie" id="link3">Tillie</a>]
```

```
In [26]: for url in soup.find_all('a'):
         print(url.get('href'))
```

```
http://example.com/elsie (http://example.com/elsie)
http://example.com/lacie (http://example.com/lacie)
http://example.com/tillie (http://example.com/tillie)
```

```
In [28]: for url in soup.find_all('a'):
         print(url.get_text())
```

```
Elsie
Lacie
Tillie
```

```
In [22]: soup.find(id="link3")
```

```
Out[22]: <a class="sister" href="http://example.com/tillie" id="link3">Tillie</a>
```

A Tag object corresponds to an XML or HTML tag in the original document:

Tag

```
In [29]: soup = BeautifulSoup('<b class="boldest">Extremely bold</b>')
         tag = soup.b
         type(tag)
```

```
Out[29]: bs4.element.Tag
```


Every tag has a name, accessible as `.name`:

Name

```
In [30]: tag.name
```

```
Out[30]: 'b'
```

If you change a tag's name, the change will be reflected in any HTML markup generated by BeautifulSoup:

```
In [31]: tag.name = "blockquote"  
tag
```

```
Out[31]: <blockquote class="boldest">Extremely bold</blockquote>
```

Attributes

```
In [33]: soup = BeautifulSoup('<b id="boldest" class="boldest">Extremely bold</b>')  
tag = soup.b  
type(tag)  
tag['id']
```

```
Out[33]: 'boldest'
```

```
In [34]: tag.attrs
```

```
Out[34]: {'id': 'boldest', 'class': ['boldest']}
```

You can add, remove, and modify a tag's attributes. Again, this is done by treating the tag as a dictionary:

```
In [35]: tag['id'] = 'verybold'  
tag['another-attribute'] = 1  
tag
```

```
Out[35]: <b another-attribute="1" class="boldest" id="verybold">Extremely bold</b>
```

```
In [37]: del tag['id']  
del tag['another-attribute']  
tag
```

```
Out[37]: <b class="boldest">Extremely bold</b>
```

```
In [39]: #tag['id'] gives error
```